Experimental Environment & Methodology

Cùng dữ liệu và phép tính, đo sau khi Spark thực thi thật.

local[2]

Một máy, 2 luồng task
JVM heap tối đa 6 GiB

Spark 3.5.7
Scala 2.12.18 / Java 17
GraphFrames 0.10.0
AQE OFF / broadcast OFF

01 **Input chung đã sẵn sàng**

Đọc Parquet và ánh xạ Long ID nằm ngoài timer.

02 **1 warmup + 3 lượt đo**

Chạy tuần tự, trộn thứ tự với seed 42. Báo cáo median và min/max.

03 **Timer bao gồm materialization**

Checkpoint trong timer. Reference và correctness ngoài timer.

4 partitions | GraphX: EdgePartition2D | GraphFrames: edges theo src | MEMORY_AND_DISK

Graph Scale & Benchmark Questions

Graph giao dịch giữ nguyên cạnh song song, không lấy mẫu.

422,734

tài khoản / vertices

4,487,124

giao dịch có hướng / edges

A **Build**

Tốn bao lâu để dựng
và materialize graph?

B **Aggregation**

Mỗi tài khoản nhận
tổng cộng bao nhiêu USD?

C **PageRank**

Chi phí cập nhật rank
qua đúng 5 vòng là gì?

16 / 16	kiểm chứng framework đạt	**24 / 24**	kiểm chứng partition đạt	Reference độc lập theo từng vertex

Graph Representation & Build

GraphX tạo cấu trúc graph chuyên dụng. GraphFrames giữ hai bảng.

GraphX

```
val v = vertices.select("id").as[Long].rdd
  .map(id => (id, 1.0))
val e = edges.select("src", "dst", "amount")
  .as[(Long, Long, Double)].rdd
  .map { case (s, t, a) => Edge(s, t, a) }
val level = StorageLevel.MEMORY_AND_DISK
val gx = Graph(v, e, 1.0, level, level)
  .partitionBy(PartitionStrategy.EdgePartition2D, 4)
  .persist(level)
gx.vertices.count()
gx.edges.count()
```

GraphFrames

```
val v = vertices
  .select(col("id"), lit(1.0).as("value"))
  .repartition(4, col("id"))
  .persist(StorageLevel.MEMORY_AND_DISK)
val e = edges.select("src", "dst", "amount")
  .repartition(4, col("src"))
  .persist(StorageLevel.MEMORY_AND_DISK)
val gf = GraphFrame(v, e)
gf.vertices.count()
gf.edges.count()
```

VertexRDD + EdgeRDD

Index và routing hỗ trợ xử lý graph.

DataFrame + DataFrame

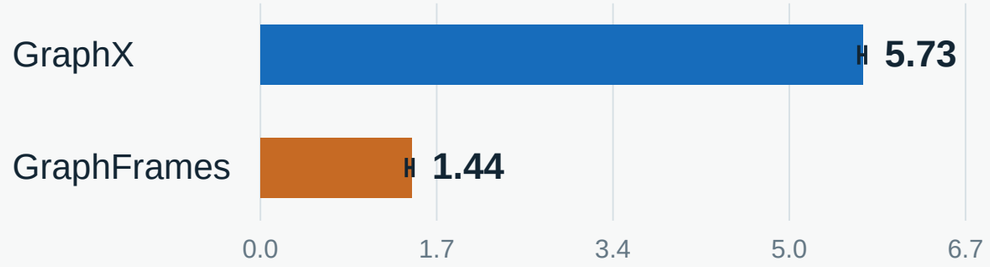
Biểu diễn bảng, thực thi qua Spark SQL.

Build Performance

GraphFrames dựng graph nhanh hơn khoảng 4 lần trong cấu hình này.

Build time

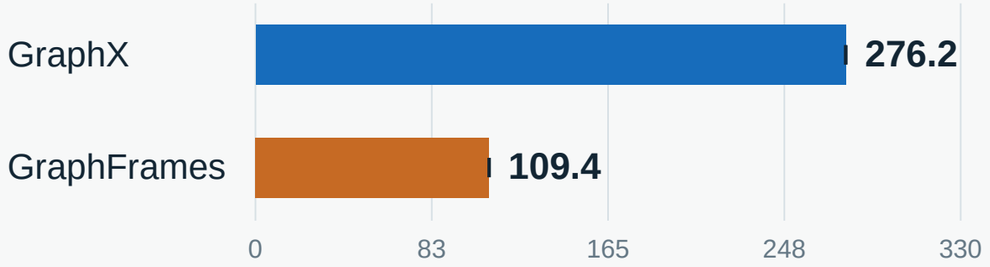
giây - thấp hơn tốt hơn



Median; thanh sai số: min/max của 3 lượt đo

Cache cuối build

MiB - footprint được giữ



Median; thanh sai số: min/max của 3 lượt đo

4.0×

GraphFrames nhanh hơn khi build

Chi phí chuẩn bị khác nhau

GraphX chuyển sang RDD, partition cạnh và dựng cấu trúc graph. GraphFrames partition và cache các bảng có sẵn.

GraphX giữ nhiều cache hơn. Đây là quan sát footprint, chưa chứng minh cache tạo ra lợi thế tốc độ.

Message Aggregation

Cùng gửi amount tới đích và cộng theo tài khoản nhận.

GraphX

```
val incoming = gx.aggregateMessages[Double](
  sendMsg = edge => edge.sendToDst(edge.attr),
  mergeMsg = (a, b) => a + b,
  tripletFields = TripletFields.None
)

// amount nằm trên cạnh
// Không yêu câù thuộc tính vertex
```

GraphFrames

```
import org.graphframes.lib.AggregateMessages
import org.apache.spark.sql.functions._

val incoming = gf.aggregateMessages
  .sendToDst(AggregateMessages.edge("amount"))
  .agg(sum(AggregateMessages.msg).as("incoming"))

// Biểu thức cột và aggregation của Spark SQL
```

Giao dịch vào B

A → B: $100

C → B: $50

SUM(amount)

Kết quả

B nhận $150

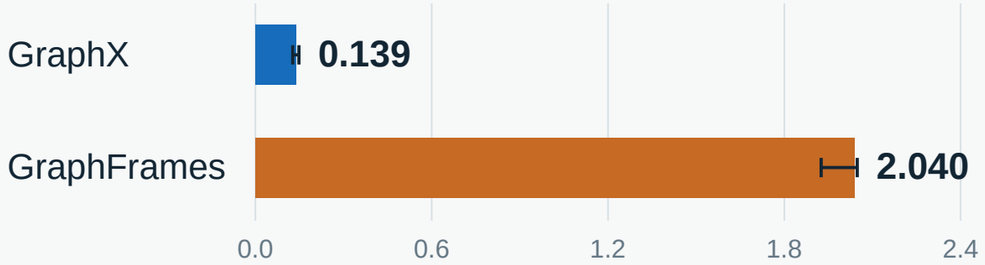
Timer gồm gửi message, cộng và materialize. Vertex không nhận message được xem là 0 khi kiểm chứng.

Aggregation Performance

Một phép cộng đơn giản vẫn thể hiện chi phí thực thi rất khác nhau.

Aggregation time

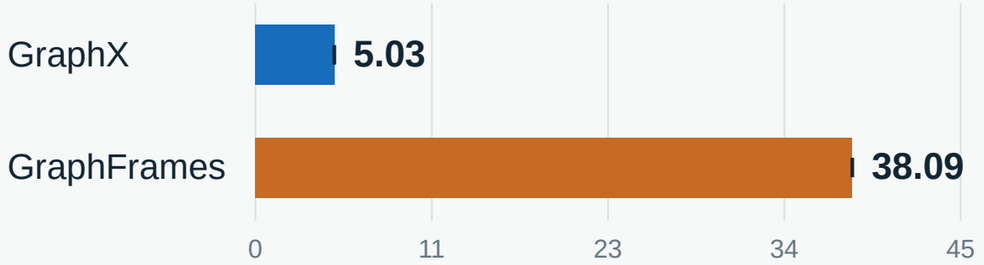
giây - thấp hơn tốt hơn



Median; thanh sai số: min/max của 3 lượt đo

Shuffle read

MiB - một lượt aggregation



Median; thanh sai số: min/max của 3 lượt đo

14.7×

GraphX nhanh hơn khi aggregate

Graph operator so với relational plan

GraphX chỉ đọc thuộc tính cạnh. Plan GraphFrames đã ghi nhận SortMergeJoin, Exchange và HashAggregate.

```
GraphFrames: join endpoints → exchange → aggregate
```

Iterative PageRank

Cùng công thức, khác cách gắn rank nguồn và cập nhật state.

$$r'(v) = (1 - d)/N + d\,[\,\Sigma_{u \to v}\, r(u)/\text{outDegree}(u) + D/N\,]$$

d = 0.85 · rank ban đầu = 1/N · 5 vòng

GraphX

```
// g: rank ở vertex, 1 / outDegree ở edge
val messages = g.aggregateMessages[Double](
  e => e.sendToDst(e.srcAttr * e.attr),
  (a, b) => a + b,
  TripletFields.Src
)
val next = g.outerJoinVertices(messages) {
  (_, _, msg) =>
    teleport + d * (msg.getOrElse(0.0) + share)
}
```

GraphFrames

```
// state: id, rank; weightedEdges: src, dst, weight
val graph = GraphFrame(state, weightedEdges)
val messages = graph.aggregateMessages
  .sendToDst(AggregateMessages.src("rank") *
    AggregateMessages.edge("weight"))
  .agg(sum(AggregateMessages.msg).as("message"))
val next = state.select("id")
  .join(messages, Seq("id"), "left")
  .select(col("id"), (lit(teleport) + lit(d) *
    (coalesce(col("message"), lit(0.0)) +
      lit(share))).as("rank"))
```

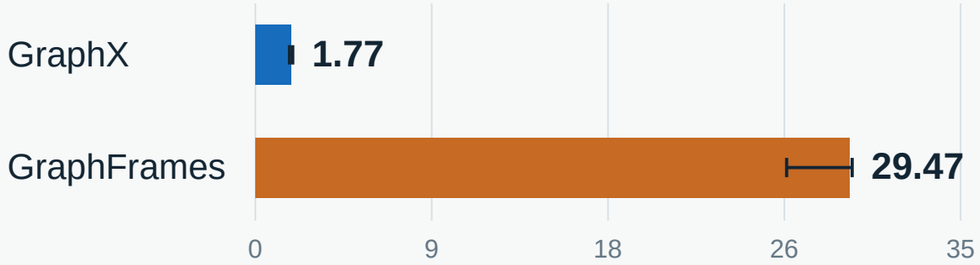
D là dangling mass. teleport = (1 − d)/N, share = D/N. Đoạn trích minh họa một vòng cập nhật.

PageRank Performance

Chi phí cập nhật lặp khuếch đại chênh lệch giữa hai implementation.

PageRank total time

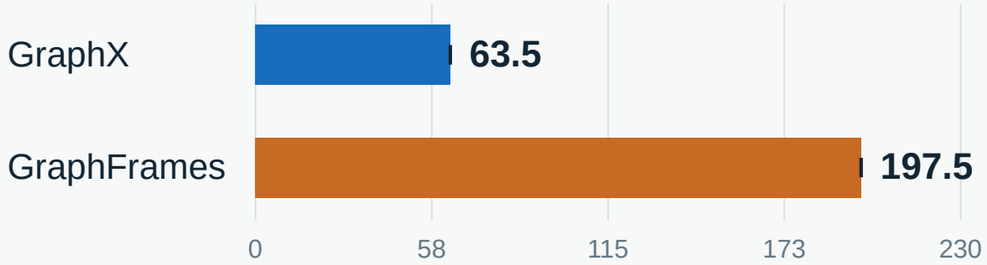
giây - setup + 5 vòng



Median; thanh sai số: min/max của 3 lượt đo

Shuffle read của 5 vòng

MiB - không gồm setup



Median; thanh sai số: min/max của 3 lượt đo

16.7×

GraphX nhanh hơn ở PR total

State và topology qua nhiều vòng

GraphX tái sử dụng cấu trúc graph khi đổi thuộc tính. GraphFrames lặp lại message aggregation và join để cập nhật bảng state.

Checkpoint I/O nằm trong timer cả hai bên. Kết quả gắn với implementation và cấu hình này.

Partition Strategies: Theory & Scala Code

Giữ nguyên GraphX và phần cứng, chỉ thay strategy và số partition.

RandomVertexCut

Hash cặp (src, dst). Baseline cùng P.
Không tối ưu trực tiếp số bản sao vertex.

EdgePartition1D

Hash theo src, gom cạnh cùng nguồn.
Có thể lệch tải khi nguồn có nhiều cạnh.

EdgePartition2D

Ánh xạ nguồn và đích vào lưới 2D.
Giới hạn replication lý thuyết ≤ 2⌈√P⌉.

GraphX: một API, ba lựa chọn

```
val strategies = Seq(
  PartitionStrategy.RandomVertexCut,
  PartitionStrategy.EdgePartition1D,
  PartitionStrategy.EdgePartition2D
)
for (p <- Seq(4, 16); s <- strategies) {
  val g = Graph(vertexRDD, edgeRDD, 1.0)
    .partitionBy(s, p)
  // Cùng cache, PageRank và cách đo
}
```

3 strategies × 2 mức P

1 warmup + 3 lượt đo / cấu hình

Cả ba đều là vertex-cut

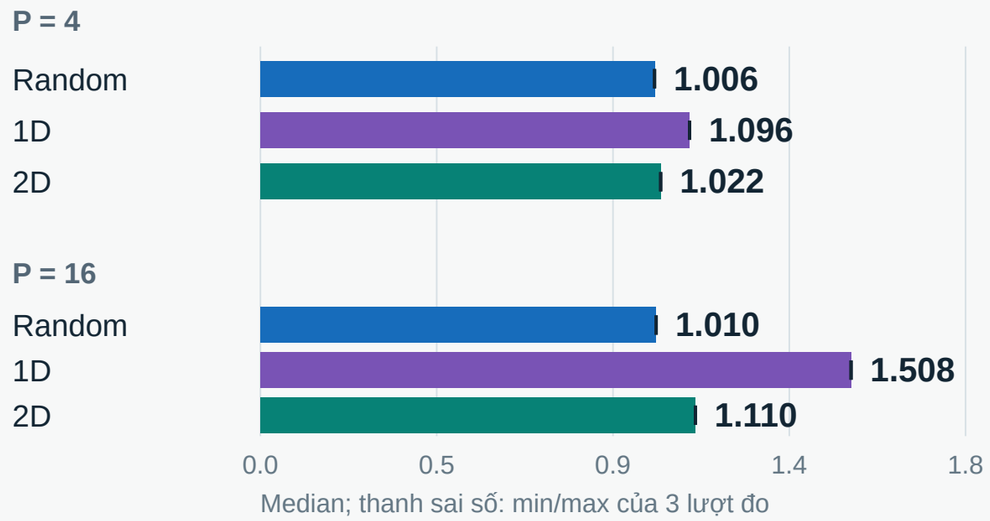
Load Balancing & Vertex Replication

Random chia cạnh đều nhất, nhưng số bản sao vertex cao nhất.

● RandomVertexCut ● EdgePartition1D ● EdgePartition2D

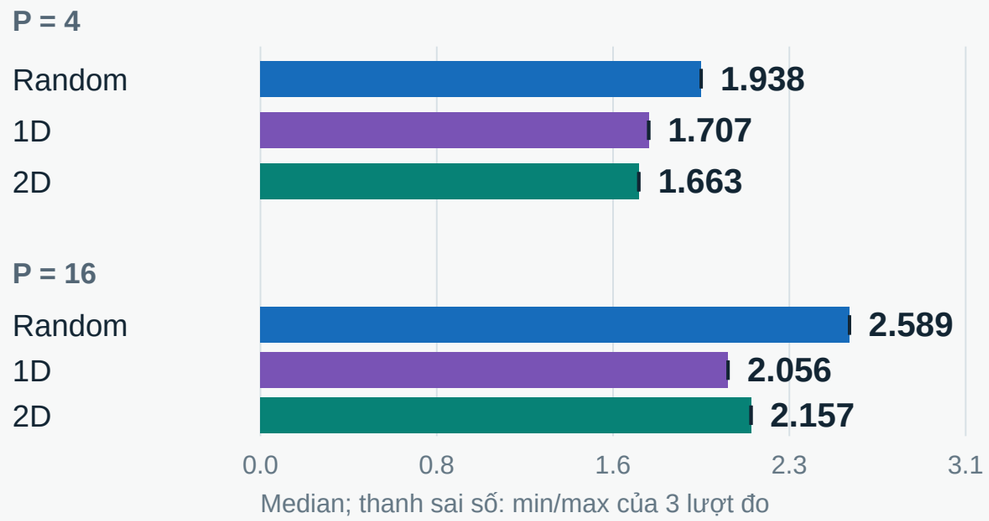
Độ lệch tải

max edges / mean edges - gần 1 là đều



Replication factor

edge partitions / vertex có cạnh



Tại P = 16

1D có max/mean 1.508, so với Random 1.010.

Đổi lại, 1D ít nhân bản hơn

2.056 so với 2.589, giảm khoảng 21%.

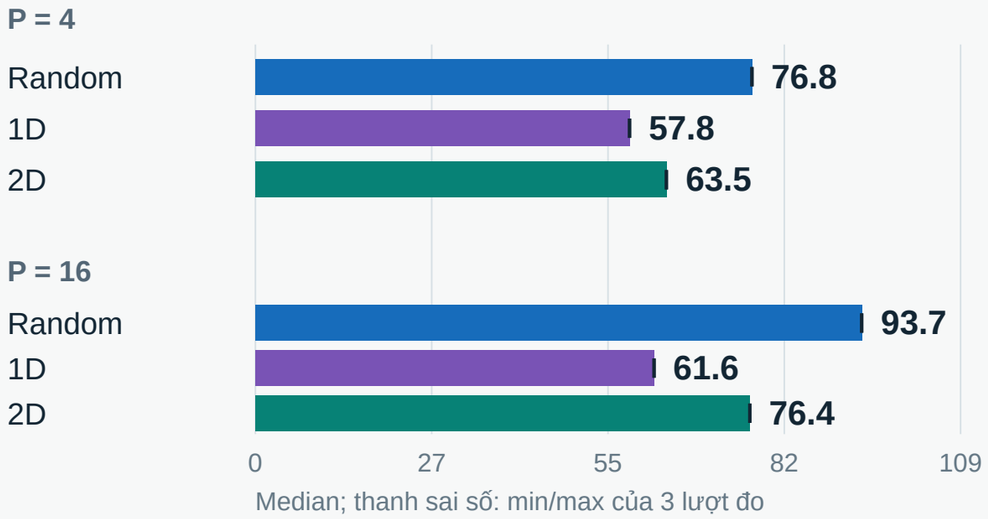
Shuffle & PageRank Performance

1D giảm shuffle rõ nhất. Shuffle local[2] chưa đo traffic giữa máy.

● RandomVertexCut ● EdgePartition1D ● EdgePartition2D

Shuffle read của 5 vòng

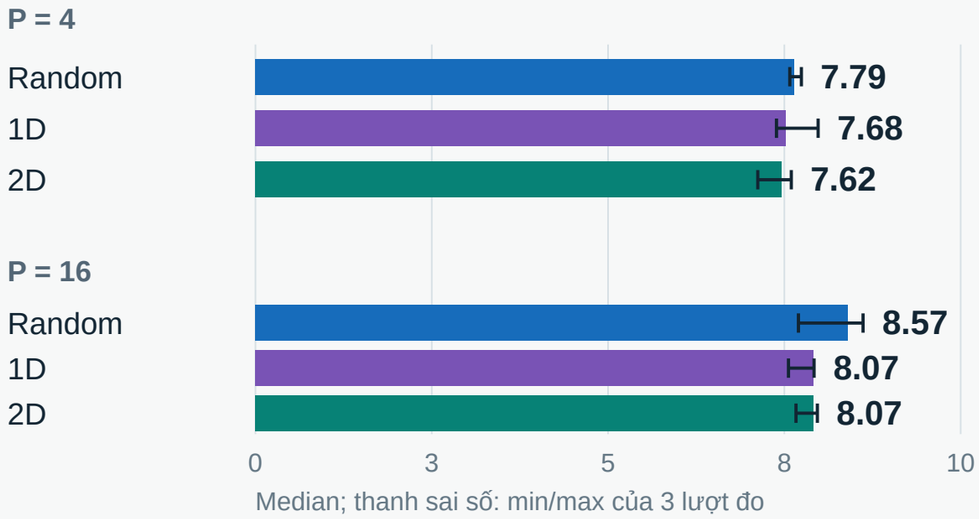
MiB - không gồm build và setup



Median; thanh sai số: min/max của 3 lượt đo

Tổng thời gian

giây - build + setup + 5 vòng



Median; thanh sai số: min/max của 3 lượt đo

P16: 1D so với Random

Read giảm 34%, tổng thời gian giảm 5.8%.

16 partitions chưa giúp chạy nhanh hơn

Số luồng vẫn là 2. Thời gian tổng tăng ở cả ba strategy.

Experimental Findings & Workload Suitability

Lợi thế phụ thuộc công đoạn và cách thực thi workload.

GraphFrames

Build nhanh hơn khoảng 4×

Phù hợp để cân nhắc khi workflow bắt đầu từ bảng và cần tích hợp SQL.
Lợi ích tích hợp là đặc tính API, chưa benchmark toàn bộ pipeline SQL.

GraphX

Aggregation 14.7×, PageRank total 16.7× nhanh hơn

Đáng cân nhắc cho workload truyền message và cập nhật graph lặp lại.
Tổng các phase đã đo: **7.59 s so với 32.90 s**, chênh lệch 4.33×.

Partitioning

Tải đều và ít shuffle là hai mục tiêu khác nhau

Random cân bằng nhất, 1D ít shuffle nhất. Chọn bằng phép đo trên workload.
P = 16 chưa cải thiện thời gian trên local[2].

> Phạm vi kết luận: HI-Small, một máy, cấu hình cố định và PageRank tự triển khai 5 vòng.